Behique: phases and progress review

Status as of 2026-10-31, covering all work through 2026-10-30 (everything is committed). For a read-through before more is built. Everything here is marked as measured, built but unmeasured, or not built.

51
commits on main, none pushed (since 2026-10-29)

567
offline tests passing in release, onboard, crawl, ui, sandbox (612 with bench tests, senses excluded; re-run today)

~$40
model spend so far of about $250 credits (estimate)

4 days
to the hackathon deadline (2026-11-04, 9 PM ET, not verified)

1. What Behique is

An autonomous release-verification and visual QA tool. You give it a known-good release and a release candidate of a web app. It records what each one shows along real click paths, finds what changed, replays every change on a fresh build to prove it is real (3 of 3 on each side), finds the exact commit that caused it, and writes an evidence-backed report. **A model proposes, code verifies, a person approves.** A bug only counts when a replayable assertion proves it, and the Ship or Hold decision is always a person's.

2. Phases at a glance

Piece	What it does	Status	Evidence
Explorer (early)	Explores an unseen app, reports defects as data assertions, replays each fresh.	**Built, n=1**	28 planted bugs on a todo app; noisy results.
Release pipeline	Build a git ref by sha, record page facts, compare good vs candidate with a control, write a test, bisect.	**Built, measured**	FreshRSS 1.29.1 vs 1.30.0: 2 real changes, 3/3 + 3/3, first bad commit 1b7178a in 8 steps.
Evidence store and report	Hash-chained evidence, labelled claims, report page, person's Ship or Hold, optional AI guess with price shown first.	**Built, used**	Report page and dashboard run end to end.
Canvas senses	Pixel diff and coordinate actions for canvas apps (Music Blocks).	**Built, measured**	Fixture: 18 of 18 detectable bugs, 3/3 reproduced. Music Blocks v3.8.0 vs master run (about $0.07).

Wave 1 and 2	Several findings per run, regions, screenshots, visual runs, set-up from the dashboard.	**Built, committed**	Share and scroll scenarios unchanged by the wiring.
Phase 0: Measure	Does set-up work on real repos?	**Measured in the sandbox**	**1 of 6** proposals pass as written; **6 of 6 verified after the repair loop** (1 to 3 rounds each, about $0.10).
Phase 1: Set-up v2	Richer repo facts, package-manager and env handling, repair loop, profile format v2.	**Built, measured**	Repair loop ran on 5 real failing repos: all verified.
Phase 2: Flow crawl	Finds click paths with no model, read-only, then compares along them.	**Built, run on 7 apps**	Reproduced the FreshRSS Share regression with zero hand-written steps (tuned on FreshRSS). Ran on six other repos, one run each; see section 7.
Phase 3: Seed data	Put realistic data in an app that needs it.	**Not built**	Depends on Phase 4 sidecars.
Phase 4: Isolation	Run untrusted repos in Docker with database sidecars.	**Built, measured on hardac2**	First-party app + Postgres, todomvc and umami (with Postgres) ran in it; see section 6. A stalled install is now killed after 8 minutes and retried once.
Check bench on six repos	Runs the whole check (crawl, compare, confirm) on each repo in the sandbox and keeps one table.	**Built, one run per repo**	5 of 6 gave confirmed changes, 1 no change; 0 failed after the noise fix. No ground truth. Section 7.
Planted-changes bench	Plants known changes in a small generated app and counts what the check finds.	**Built, measured**	8 of 8 visible changes found, 7 of 8 confirmed, 0 false positives.

Phase 5: Cloning	Clone a repo from the dashboard after a person approves the exact command.	**Built, previewed only**	Refusals tested; nothing cloned through it. Your own GitHub repos only.

3. The result that matters

With no hand-written steps, the crawler plus compare reproduced the FreshRSS Share regression: the share button's `data-url` and `data-title` are now URL-encoded. The crawl visited 30 states and produced 37 steps. The two changes were confirmed 3 of 3 on the good build and 3 of 3 on the candidate. Bisect found commit `1b7178a` in 8 steps, the same commit the hand-written case gives. The same comparison started from the dashboard returned HOLD with 2 reproduced changes and a screenshot.

> **Read this as:** the mechanism works. It is **not** evidence that it works on any app, because FreshRSS is the app the crawler was tuned on. The first runs of the crawl returned `fails_both` because recorded steps shared state across flows. The fix (replay each flow alone, read-only, cookies and storage cleared between flows) is what made it honest.

4. What was built overnight (2026-10-30)

Built with Muse (Meta's model) from written specs, each file read and corrected by hand. Seven commits, all offline-tested, nothing pushed.

Commit	Contents
`894f337`	inspect-more, toolchain, repair modules (specs 13 to 15)
`a7b57bd`	Profile format v2: env, static root, build steps, installs with scripts disabled
`4de1d99`	Build runner: output tail, early-exit detection, whole process group is stopped
`b0f1cd0`	The read-only crawler and `check-app` (specs 16 to 18)
`a21da41`	Read-only, per-flow replay in confirm, facts, compare, findings, shots
`13dcaa4`	Clone flow and crawl runs in the dashboard server (spec 19)
`ea6e14a`	Set-up bench, baseline and overnight log, STATUS

Real bugs found in the process: Muse's code had a test design flaw, two contradictory tests, a wrong ANSI pattern and a risk filter that treated any row containing "post" as dangerous. The build runner had a genuine bug: a dev server started through yarn or npm left a child process holding its port after a stop.

5. Safety: what was blocked and why

Twice the six-repo run was denied by Claude Code's permission system as "[Code from External]": it will not let me install dependencies and start dev servers from freshly cloned third-party repos unattended,

because that runs the repo's code on your Mac. I did not work around it. That is why Phase 4 was built: the same runs, executed in Docker on hardac2, were accepted, and nothing from those repos ran on the Mac.

- Behique's own safeguards: installs run with scripts disabled; commands come from an allowlist; clone and set-up run only with the fingerprint of what a person reviewed; the crawler sends only GET and aborts cross-origin requests.
- I read each proposed command list before running it. Everything still runs only when `BEHIQUE_SANDBOX` is set; without it, profile apps run as normal host processes, as before. A mistyped value is an error, never a silent fallback to the host.

6. Phase 4: untrusted repos run in Docker, with database sidecars

Decision: one Docker runner with two targets, switched by `BEHIQUE_SANDBOX`. hardac2 over ssh is the real target (x86_64, Docker 29.9.2, 24 cores, KVM available); OrbStack on the Mac is the local option (arm64, daemon was off, not used today). Full details: `docs/sandbox.md`.

How it works

- Each check gets an **internal network** (no outside access) and an open one. Sidecars (Postgres, MySQL, MariaDB, Redis, MongoDB) and the app run on the internal one; install steps also join the open one for the package registry.
- The app is reached by its container address through a tunnel, so everything else in Behique still sees `http://127.0.0.1:port`. Nothing is published to hardac2's network.
- Every container: all capabilities dropped, no-new-privileges, read-only root, non-root user (databases excepted, with a minimal capability set), memory, CPU and process limits, deleted after the check. Lifecycle scripts stay disabled on installs.
- Every docker argument is built by code from validated parts (`docker-args.mjs`); nothing from a repo, a model or a page can become a docker flag. This matters because the docker group is root-equivalent on hardac2.

Piece	Made by	Result
docker-args (policy)	Muse spec 20, then my review fixed: root users via gid, `..` paths, a missing service-user rule	**29 tests**
services (sidecars)	Muse spec 22; I fixed redis and mongo needing the same capabilities as the databases	**14 tests**
sandbox (lifecycle)	Muse spec 21; I fixed broken ANSI patterns that deleted letters, and stale-run cleanup that missed orphaned networks and volumes	**16 tests**
runner, tunnel, backend, launcher, clean	By hand	**tests with local sockets and fakes**

Measured on hardac2

Case	Result

First-party app + Postgres sidecar	**Up in 3.6 s. The app reached `db:5432` by name, ran as uid 1000, and an outbound connection to the internet failed (ENETUNREACH). Nothing left behind.**
todomvc (static)	**VERIFIED end to end (build, pictures, noise floor).**
html5-boilerplate	**Built (`npm ci` + build) and served; the "app is alive" check failed because the page is almost blank by design.**
umami v3.3.1 + Postgres	**Install, build and `next start` ran in the sandbox; `/login` answered 200. Needed 10 GB, the full Node image and a corrected start command.**
Repair loop on the failing repos	**nextjs-blog (2 rounds), vitepress (2), excalidraw (3) and umami (3, the loop added the Postgres sidecar itself) all VERIFIED; html5-boilerplate passes once the alive check accepts a mostly white page with text. Changed commands were approved only within a narrow policy (same programs, env, sandbox services); none was refused.**

Bugs the real runs found (all fixed, each with a test)

- Docker copies the image folder's owner (root) onto a still-empty volume at first mount, silently undoing the permission fix. Fixed with `nocopy`.
- `/tmp` must allow execution: yarn Berry writes launchers there. Yarn Berry also refuses `--ignore-scripts`; scripts are now turned off the way each yarn version accepts.
- Dev servers that listen only on localhost were unreachable from outside the container. A small launcher inside the container relays to them.
- The default Node image lacked openssl (Prisma needs it); `next build` was killed at 4 GB (needs about 10 GB); the model's `run start -- -p {port}` breaks with pnpm and yarn Berry.
- An ssh tunnel through the shared connection exited at once; it now uses its own connection.

Limits: plain Docker (runc) is not a hard wall; before arbitrary repos, move to Firecracker or gVisor. The sidecar proposer missed umami's Postgres (its compose file names a service `db` with no hint). The repair loop now ran for real (see above) under a narrow standing approval; anything outside it (a new program, npx or dlx, scripts on) stops for a person. Phase 3 (seed data) is not built.

7. Checks on six real repos (2026-10-30)

`bench/check-bench.mjs` runs the full check (crawl the good version, compare both along the crawled paths, confirm each change as a test run 3 times on each side) on each repo in the sandbox on hardac2, and writes one table. The crawl is also graded (good, thin, degenerate, failed) so a result can be read with its coverage in mind.

Repo	Result	Crawl	What it found
html5-boilerplate	**no change**	degenerate (landing page only)	Says little: only one page was reachable.
excalidraw	**5 of 5 confirmed**	thin (12 pages)	Canvas app, so the crawl reaches little of it.

nextjs-blog	**5 of 5 confirmed**	good (20 pages)	1,223 facts had to be masked as noise.
vitepress	**1 of 1 confirmed**	good (19 pages)	The version label: true but trivial. 40 facts masked.
umami	**3 of 3 confirmed**	thin (2 pages, login wall)	Button `data-slot` attributes and a React-generated id (`_r_1_` to `_r_2_`): reproducible, means little.
todomvc	**5 of 5 confirmed**	good (35 pages)	66 facts masked; a rotating four-slot testimonial needed 5 extra recordings.

> **Read this as:** "confirmed" means the test passes on the good version and fails on the candidate every time. It does **not** mean the change matters, and there is no ground truth for these repos. Each result is one run.

What it took

- The first full run gave **1 confirmed, 1 no change, 4 failed**. Three failed because the good version compared with itself still differed (a rotating testimonial, a locale that flipped at one step, a dialog open in one recording). One was an install that hit the 15-minute limit.
- Fix: facts the control disagrees on now join the noise mask (an element noisy at three or more steps is masked at every step), and a fresh control must come back clean, up to 5 extra recordings. **Cost: a real change inside a masked fact cannot be seen.** This needs a second look.
- Also fixed: a 3 s navigation limit that left excalidraw with 0 pages (now 30 s, and the reason is logged); a crawl with nothing to click now loads `/`; text findings compare the element's own text exactly (`own_text`) because a substring check made text changes unconfirmable; a stalled-install watchdog (8 minutes without output, one retry on a fresh container, not yet seen to trigger on a real repo).
- Run times were 1 to 38 minutes per repo, mostly installs and builds.

Planted changes

`bench/planted-bench.mjs` (a small generated app, 9 single changes plus a comment-only change): **8 of 8 visible changes found, 7 of 8 confirmed as a test, 0 false positives** on the comment-only change. The one found but not provable is an added `disabled` attribute (an added fact has no value to assert against); a background-colour change is left to the picture check.

8. What I would check for "tip top shape"

Run these (all safe, nothing is installed from third-party repos)

```
cd ~/dev/behique
git status                      # clean: everything is committed
git log --oneline | head -12    # check bench, noise learning, watchdog and demo script
commits are on top
node --test src/release/*.test.mjs src/release/onboard/*.test.mjs \
  src/release/crawl/*.test.mjs src/release/ui/*.test.mjs \
  src/release/sandbox/*.test.mjs   # expect 567 pass, 0 fail
```

Look at by hand

- `out/check-bench/` (the saved tables of the six-repo runs; the last row of each repo is the final result) and `docs/STATUS.md`, section 'Checks on real repos'.
- `docs/overnight-log.md` and `docs/onboarding-baseline.md`: the honest account of the overnight run and what is unmeasured.
- `docs/STATUS.md`: the full list of what is built and the known limits.
- `docs/specs/wave3/` (specs 13 to 22): what each Muse module was told to do, in plain English.
- The dashboard on port 8790: pick Check, then Behaviour, and look at the crawl result and the clone panel.

Known weak spots

- The crawler was tuned on one app and has now run on six more, once each, with thin or degenerate coverage on three of them (excalidraw, umami, html5-boilerplate). No login or seeded data, so apps behind a login are out of reach.
- The repair and check results are one run per repo on six public repos we picked; not repeated, not a success rate.
- Noise learning hides facts (nextjs-blog 1,223, todomvc 66, vitepress 40); a real change inside them would be missed.
- `~/dev/behique-ui/behique-live.html` (the dashboard page) is **not under git**; backups exist only in a scratch folder.
- A Ship or Hold decision's name is typed in, so it is attribution by honour, not identity.
- The demo script was updated on 2026-10-30 (new beat 10 shows the six-repo results as saved runs, not live); the live beats still use hand-written FreshRSS steps.

9. Open questions for you

- **Hackathon rules:** the deadline (8 or 9 PM ET), track, video length limit and whether Muse Code is expected are all unverified. Please paste them from the confirmation email or official page.
- **What next, with four days left:** the demo script is updated, so the options are rehearse and record, repeat the six-repo runs to see how stable they are, or build Phase 3 seed data and login so umami-type apps can be crawled.
- **Credits:** Meta API about $146 left of $150; Baseten about $70 of $100 (estimates). Canvas work was allowed $10 and has used about $2. The Phase 4 Muse builds cost about $0.35 in all (three specs, one rebuilt).

Sources: docs/STATUS.md, docs/overnight-log.md, docs/onboarding-baseline.md, out/check-bench, git log. Numbers marked estimate come from those notes and were not re-measured today, except the test counts and commit count.